

TASK – 1

MULTI-AGENT SYSTEM USING LANGGRAPH OR LANGCHAIN AGENTS

Submitted By/-
Gollapally Sai Archan
22d41a6637@gmail.com
6303043817

1. ABSTRACT

AgentIQ is a LangGraph-based multi-agent system for generating structured company intelligence reports using coordinated agent workflows. The system consists of independent agents operating on a shared state, where a Data Collector Agent retrieves and structures market data and an Analyst Agent generates analytical insights. The architecture emphasizes modularity, deterministic execution, and stable data contracts, enabling seamless integration of data sources and analytical components.

The system supports both offline (mock) and API-driven execution, ensuring reproducibility and fault tolerance. A Streamlit interface provides interactive visualization of results. AgentIQ demonstrates practical application of agentic AI principles, including graph-based orchestration, tool abstraction, and scalable agent design, making it suitable for educational use and extensible real-world implementations.

2. INTRODUCTION

The rapid evolution of artificial intelligence has driven a transition from monolithic AI pipelines to agentic systems, where multiple autonomous agents collaborate to perform complex tasks. Traditional approaches often tightly couple data acquisition, processing, and analysis, reducing scalability and flexibility. Agent-based architectures address these limitations by decomposing problems into specialized agents that interact through a shared state, improving modularity, maintainability, and system transparency.

AgentIQ is a LangGraph-based multi-agent company intelligence system that demonstrates this agentic paradigm in practice. The system orchestrates independent agents using a graph-based execution model, enabling deterministic control flow and clear state transitions. A Data Collector Agent retrieves and structures market-related information, while an Analyst Agent processes this data to generate insights on market trends and potential risks. The architecture is designed to be API-agnostic and fault-tolerant, supporting both mock and real data sources. A Streamlit-based interface provides interactive report generation, making AgentIQ a scalable, extensible, and educational example of modern agentic AI system design.

3. EXISTING SYSTEM

In the existing system, company intelligence and market analysis are typically generated using **static or rule-based approaches** without relying on large language models or external generative AI APIs such as Gemini. Data collection is performed using predefined scripts, mock datasets, or traditional financial APIs that return raw numerical and textual information. The analysis process is often tightly coupled with the data retrieval logic, resulting in monolithic workflows where data processing and reasoning occur within a single execution pipeline.

Without the use of Gemini API keys, analytical insights are derived through **manual heuristics, basic statistical methods, or hardcoded logic**. These systems lack contextual reasoning, adaptive insight generation, and natural language explanations. Additionally, the absence of agent-based orchestration limits scalability, as adding new analytical capabilities requires modifying existing code rather than extending the system modularly. While such systems are cost-effective and easy to deploy, they provide limited intelligence, minimal interpretability, and reduced flexibility compared to agentic AI-based architectures.

4. PROPOSED SYSTEM

The proposed system enhances the existing company intelligence framework by integrating the **Gemini Generative AI API** to enable advanced reasoning, contextual understanding, and natural language insight generation. Unlike traditional systems that rely solely on static logic or numerical analysis, the proposed approach leverages Gemini's large language model capabilities to interpret structured and unstructured data in a more human-like and adaptive manner. This integration significantly improves the quality, depth, and usability of generated intelligence reports.

In the proposed system, the overall architecture is designed as a **multi-agent, agentic AI system** orchestrated using LangGraph. Each agent operates independently while sharing a common state, allowing seamless collaboration. The Data Collector Agent gathers market-related data such as stock prices, trends, and relevant company news from APIs or other data sources. This structured data is then passed to downstream agents through a shared state mechanism, ensuring consistency and traceability across the workflow.

The core enhancement in the proposed system is the **Analyst Agent powered by the Gemini API**. This agent utilizes Gemini's natural language reasoning capabilities to analyze collected market data, identify patterns, assess trends, and generate meaningful insights. Gemini enables the system to go beyond numerical outputs by producing explanatory summaries, contextual interpretations, and qualitative assessments of market behavior. This allows the system to incorporate factors such as sentiment, growth indicators, and potential risks that are difficult to capture using traditional rule-based approaches.

The proposed system is designed to be **API-agnostic and fault-tolerant**, allowing Gemini integration to be enabled or disabled as required. When Gemini API keys are available, the system performs real-time reasoning and insight generation. In scenarios where API access is unavailable, the system gracefully falls back to mock or heuristic-based logic, ensuring uninterrupted execution. This flexibility supports reproducibility in academic environments while enabling advanced intelligence generation in real-world deployments.

Additionally, the system includes a **Streamlit-based user interface** that presents Gemini-generated insights in a clear and interactive manner. Users can input a company name and receive a comprehensive intelligence report consisting of market data, predicted trends, and Gemini-powered analytical summaries. The modular agent-based design ensures that future enhancements, such as advanced prediction models or additional reasoning agents, can be integrated without modifying the core architecture.

5. SYSTEM REQUIREMENTS

This section describes the hardware and software requirements necessary to develop, deploy, and run the AgentIQ multi-agent company intelligence system. The system is designed to be lightweight and platform-independent, ensuring compatibility with commonly available computing resources.

5.1 Hardware Requirements

The AgentIQ system does not require high-end hardware, as most processing is handled efficiently through modular agents and optional external APIs. The minimum hardware requirements are sufficient for smooth execution and testing.

Minimum Hardware Requirements:

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB
- Storage: 5 GB free disk space
- System Type: 64-bit architecture
- Internet Connection: Required for optional API-based data retrieval and Gemini integration

Recommended Hardware Requirements:

- Processor: Intel Core i5 / AMD Ryzen 5 or higher
- RAM: 8 GB or more
- Storage: SSD with at least 10 GB free space
- Display: Standard monitor with minimum 1366×768 resolution

5.2 Software Requirements

The system is implemented using Python and modern AI development frameworks. All software components are open-source and widely supported.

Operating System:

- Windows 10 / 11

- Linux (Ubuntu 20.04 or later)
- macOS (latest stable version)

Programming Language:

- Python 3.9 or higher

Frameworks and Libraries:

- Streamlit – for user interface development
- LangGraph – for multi-agent orchestration
- LangChain (core components) – for agent tooling
- NumPy – for numerical operations
- Optional APIs:
 - Gemini API (for generative AI-based analysis)
 - Financial data APIs (for real-time stock data)

Development Tools:

- Code Editor / IDE (VS Code, PyCharm, etc.)
- Web Browser (Chrome, Edge, or Firefox)
- Virtual Environment (venv)

5.3 Additional Requirements

- API keys for Gemini or external data services
- Stable internet connection for real-time analysis
- Basic knowledge of Python programming and AI workflows

6. IMPLEMENTATION

Project Structure

Soulpage-genai-assignment-saiarchan/

```
|
|
|— agents/
|   |— data_collector.py
|   |— analyst.py
|
|
|— graph/
|   |— orchestrator.py
|
|
|— tools/
|   |— market_data_tool.py
|— app.py
|— main.py
|— requirements.txt
|— README.md
|— .env
```


analyst.py

```
import os

from google import genai

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

def analyst_agent(data):

    prompt = f"""

    You are a financial analyst.

    Based on the company data below, generate:

    1. Key Insights

    2. Growth Opportunities

    3. Risk Factors

    Company Data:

    {data}

    """

    response = client.models.generate_content(

        model="models/gemini-2.5-flash",

        contents=prompt

    )

    return response.text
```

data_collector.py

```
from langchain_core.tools import tool

from tools.market_data_tool import fetch_company_data

@tool

def data_collector_agent(company: str):
```

```
"""Collects market data and recent news for a company"""
```

```
return fetch_company_data(company)
```

orchestrator.py

```
from langgraph.graph import StateGraph
```

```
from agents.data_collector import data_collector_agent
```

```
from agents.analyst import analyst_agent
```

```
from typing import TypedDict
```

```
class AgentState(TypedDict):
```

```
    company: str
```

```
    market_data: dict
```

```
    analysis: str
```

```
def collect_data(state: AgentState):
```

```
    company = state["company"]
```

```
    state["market_data"] = data_collector_agent.invoke(company)
```

```
    return state
```

```
def analyze_data(state: AgentState):
```

```
    state["analysis"] = analyst_agent(state["market_data"])
```

```
    return state
```

```
def build_graph():
```

```
    graph = StateGraph(AgentState)
```

```
    graph.add_node("collector", collect_data)
```

```
    graph.add_node("analyst", analyze_data)
```

```
    graph.set_entry_point("collector")
```

```
    graph.add_edge("collector", "analyst")
```

```
graph.set_finish_point("analyst")
```

```
return graph.compile()
```

market_data_tool.py

```
def fetch_company_data(company: str):
```

```
    """
```

```
    Dummy market data tool (replace with real API if needed)
```

```
    """
```

```
    return {
```

```
        "company": company,
```

```
        "stock_price": "₹3,245",
```

```
        "market_trend": "Bullish",
```

```
        "news": [
```

```
            "Company announced strong quarterly earnings",
```

```
            "Expansion into new AI-driven services",
```

```
            "Positive analyst rating upgrades"
```

```
        ]
```

```
    }
```

main.py

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
from graph.orchestrator import build_graph
```

```
if __name__ == "__main__":
```

```
    app = build_graph()
```

```
company_name = "Infosys"

result = app.invoke({
    "company": company_name
})

print("\n Company Intelligence Report\n")

print("Company:", company_name)

print("\n Market Data:\n", result["market_data"])

print("\nAnalysis:\n", result["analysis"])
```

requirements.txt

langchain>=0.1.16

langgraph>=0.0.30

google-genai

python-dotenv

streamlit

.env

GEMINI_API_KEY=AIzaSyCQYwfOz6p1Qm8xc63xZ6h54HKMvJBRwLg

app.py

```
import streamlit as st

from graph.orchestrator import build_graph

st.set_page_config(
    page_title="Automated Company Intelligence Using Agentic AI and LangGraph",
    page_icon="🤖",
    layout="centered"
```

)

st.markdown(

"""

<style>

.stApp {

background: linear-gradient(135deg, #e0f7ff, #cceeef, #b3e5ff);

color: #0b1f33;

}

.card {

background-color: rgba(255, 255, 255, 0.08);

padding: 20px;

border-radius: 15px;

margin-bottom: 20px;

box-shadow: 0 8px 32px rgba(0,0,0,0.3);

}

h1, h2, h3 {

color: #00f5d4;

}

</style>

""",

unsafe_allow_html=True

)

st.markdown("<h1 style='text-align:center;'>🚀 AgentIQ</h1>", unsafe_allow_html=True)

st.markdown("<h3 style='text-align:center;'>Automated Company Intelligence Using Agentic AI and LangGraph</h3>", unsafe_allow_html=True)

```

st.write("")

company = st.text_input(" Enter Company Name", "Infosys")

if st.button(" Generate Intelligence Report"):

    app = build_graph()

    result = app.invoke({"company": company})

    st.markdown("<div class='card'>", unsafe_allow_html=True)

    st.markdown("### Market Data (Data Collector Agent)")

    st.write(f" **Stock Price:** {result['market_data']['stock_price']}")

    st.write(f" **Market Trend:** {result['market_data']['market_trend']}")

    st.markdown(" **Recent News:**")

    for news in result["market_data"]["news"]:

        st.write(f"• {news}")

    st.markdown("</div>", unsafe_allow_html=True)

    st.markdown("<div class='card'>", unsafe_allow_html=True)

    st.markdown("### Analysis & Insights (Analyst Agent)")

    st.write(result["analysis"])

    st.markdown("</div>", unsafe_allow_html=True)

    st.markdown(

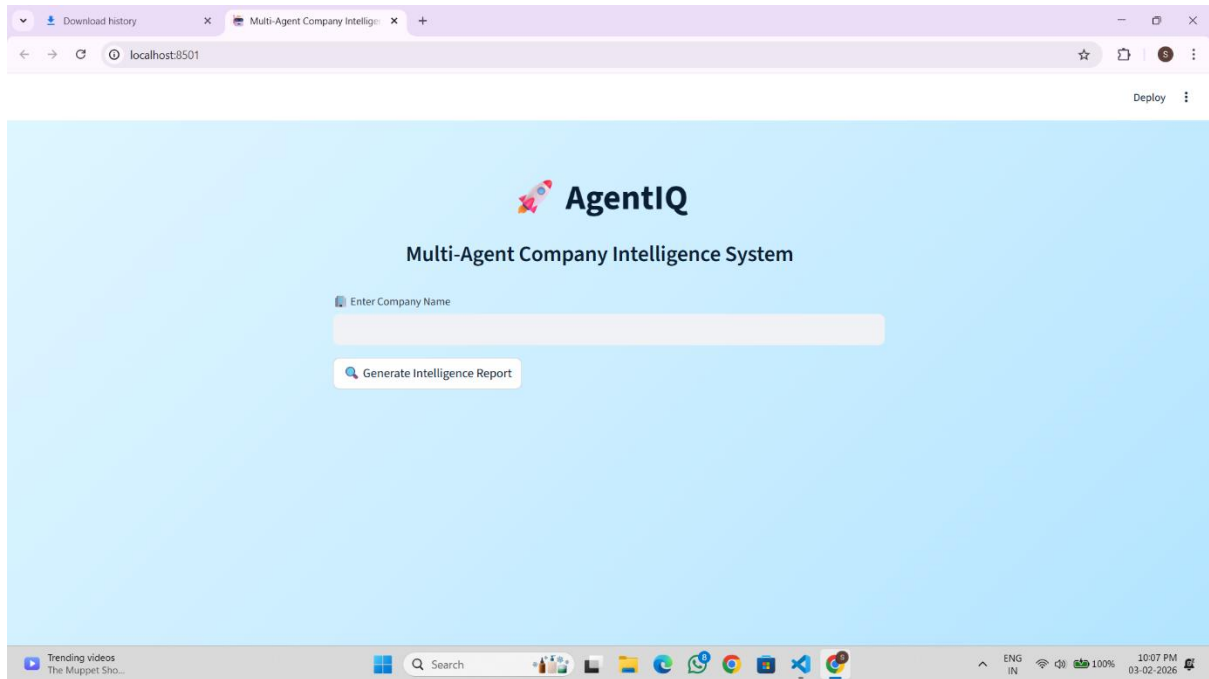
        "<p style='text-align:center; opacity:0.7;'>Powered by LangGraph • Agentic AI</p>",

        unsafe_allow_html=True

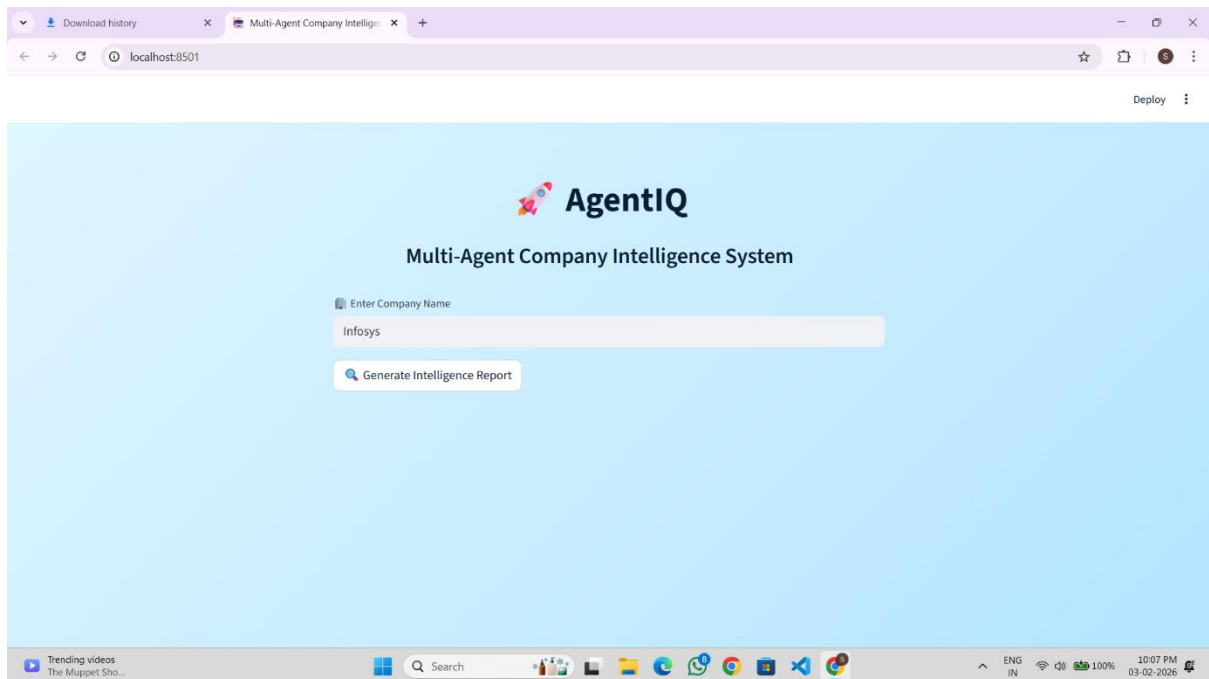
    )

```

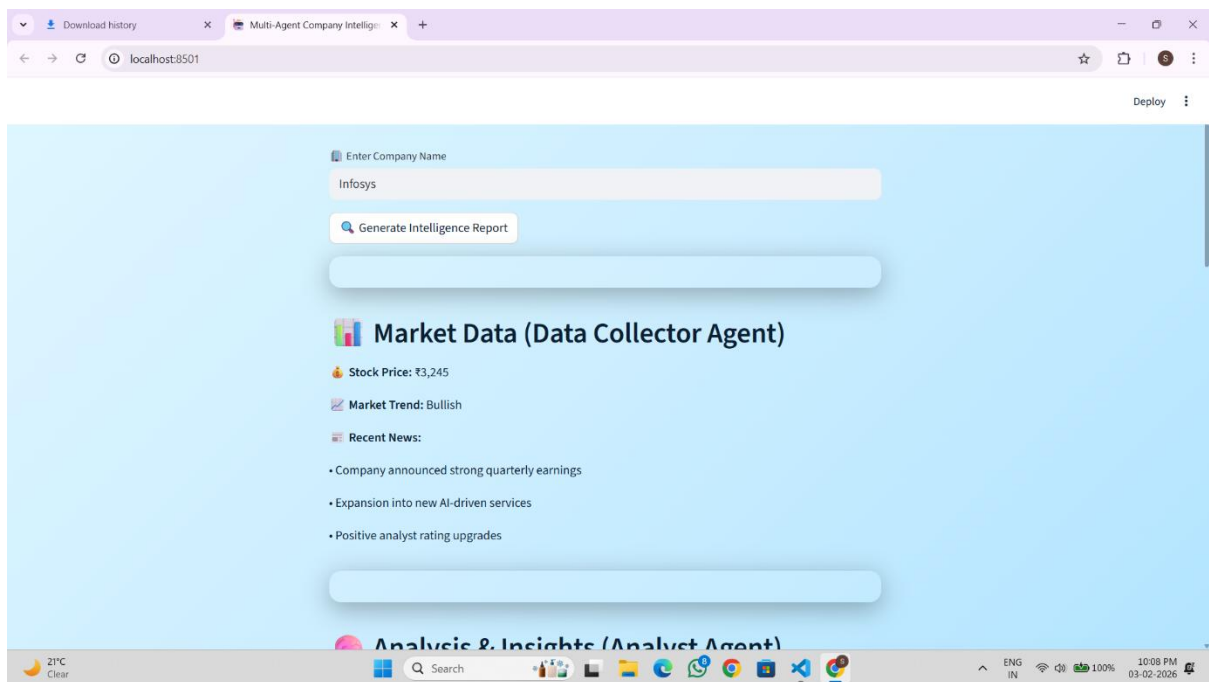
7. RESULT



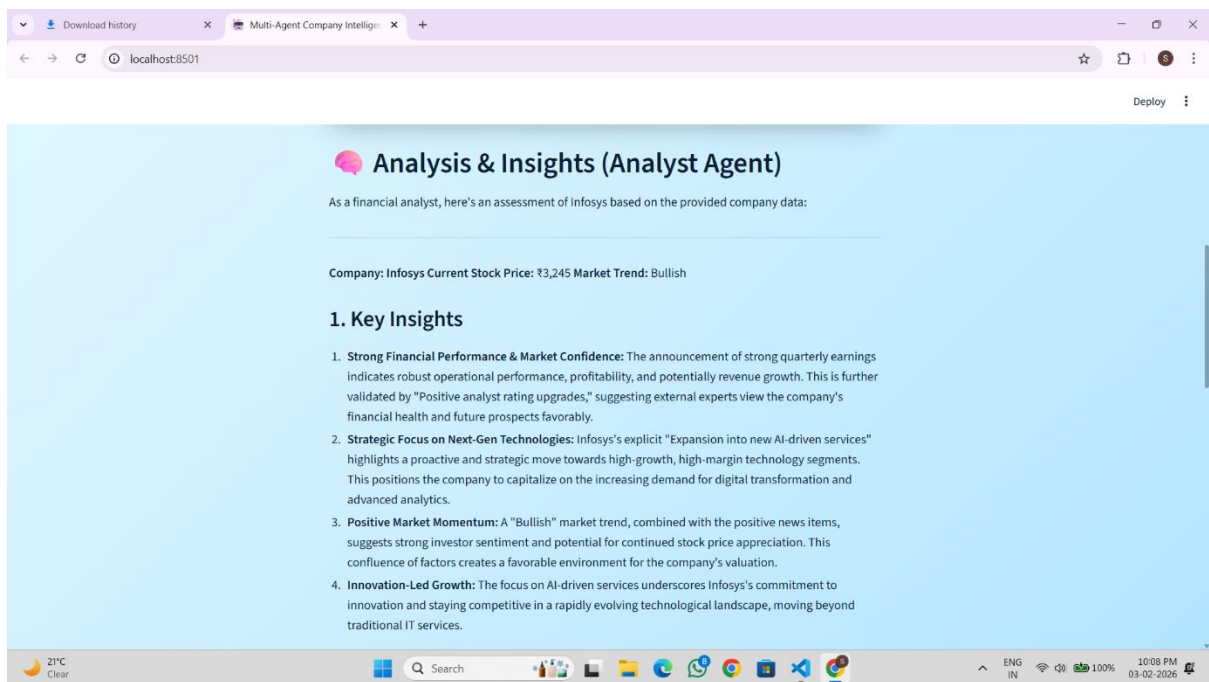
Home page



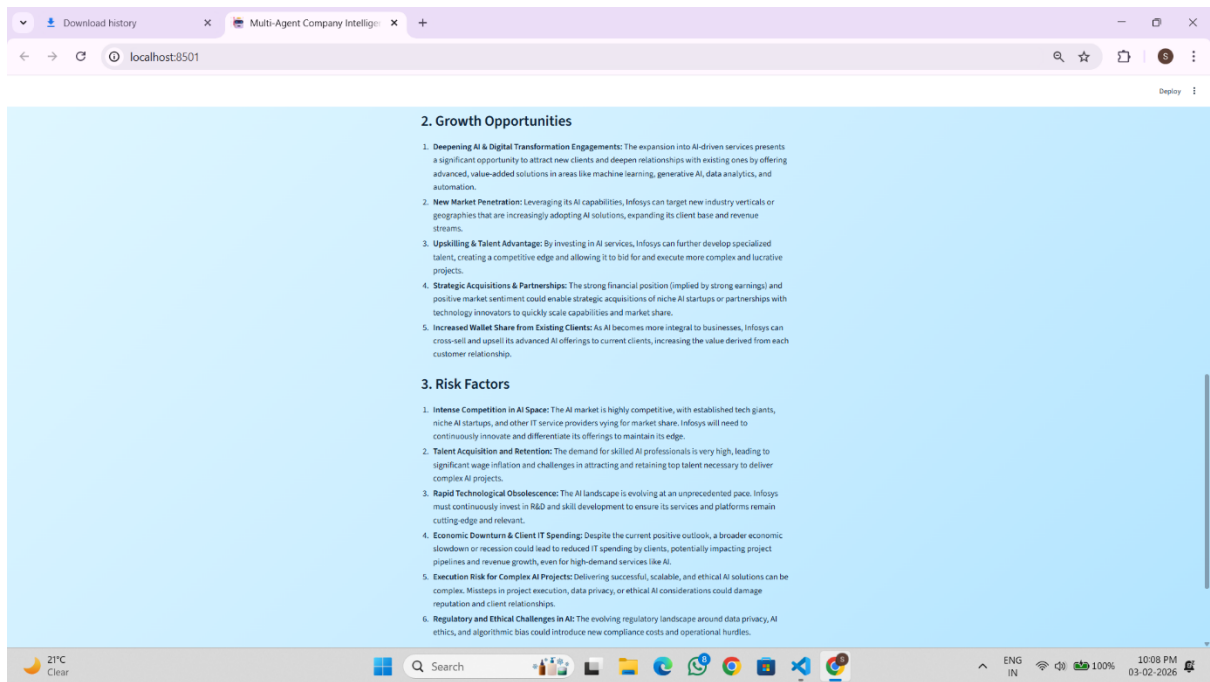
Company Name



Market Data



Company Analysis and Insights 1



Company Analysis and Insights 2

8. CONCLUSION

The AgentIQ project successfully demonstrates the design and implementation of a modern **agentic AI system** for generating structured company intelligence. By leveraging a multi-agent architecture orchestrated through LangGraph, the system effectively separates data collection, analysis, and presentation into independent yet collaborative components. This modular design improves scalability, maintainability, and clarity of execution compared to traditional monolithic approaches.

The integration of generative AI through the Gemini API further enhances the system by enabling contextual reasoning, natural language insights, and adaptive analysis. Even in the absence of external APIs, the system maintains reliable functionality through mock and heuristic-based logic, ensuring fault tolerance and reproducibility. Overall, AgentIQ serves as a strong educational and portfolio-ready example of agent-based AI design, highlighting the practical advantages of graph-based orchestration and extensible architectures in intelligent decision-support systems.